

# 04

## Liste i petlje

Ista obrada za stotine i hiljade sekvenci, bez kopiranja koda

# Kad promenljivih postane previše

```
# poglavlje 3: promenljiva za svaku sekvencu
seq_1 = "ATCGTACGATCGATCGCTAGACGTATCG"
seq_2 = "actgatcgacgatcgatcgatcacgact"
seq_3 = "ACTGAC-ACTGT--ACTGTA----CATGTG"
# i skoro ista naredba tri puta
output_1 = open(header_1 + ".fasta", "w")
output_2 = open(header_2 + ".fasta", "w")
output_3 = open(header_3 + ".fasta", "w")
```

A šta sa 300 ili 3000 sekvenci? Ili kad ne znamo unapred koliko ih ima?

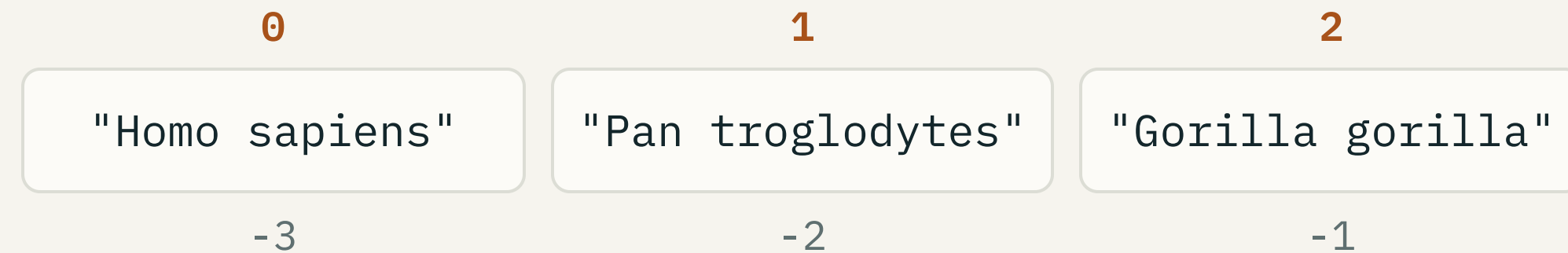
## Lista

Jedna promenljiva čuva mnogo vrednosti.

## Petlja

Ista naredba za svaki element: ponavljanje sa varijacijom.

# Liste: mnogo vrednosti u jednoj promenljivoj



Svaka stavka je element. Indeksi kreću od 0, a negativni broje od kraja: `apes[-1]` je poslednji element.

```
apes = ["Homo sapiens", "Pan troglodytes", "Gorilla gorilla"]
conserved_sites = [24, 56, 132]
print(apes[0])                                # Homo sapiens
chimp_index = apes.index("Pan troglodytes")    # 1
ranks = ["kingdom", "phylum", "class", "order", "family"]
lower_ranks = ranks[2:5]                       # ['class', 'order', 'family']
```

# Dodavanje elemenata i spajanje lista

```
# apes: lista sa tri vrste
print("There are " + str(len(apes)) + " apes")
apes.append("Pan paniscus")
print("Now there are " + str(len(apes)) + " apes")
```

There are 3 apes

Now there are 4 apes

**append** menja listu na kojoj je pozvan.

```
# apes ponovo ima tri elementa
monkeys = ["Papio ursinus", "Macaca mulatta"]
primates = apes + monkeys
print(str(len(apes)) + " apes")
print(str(len(monkeys)) + " monkeys")
print(str(len(primates)) + " primates")
```

3 apes

2 monkeys

5 primates

+ pravi novu listu, a originali ostaju isti.  
**extend** dodaje elemente druge liste i menja listu.

# Menjanje liste

| Alat                         | Šta radi                      | Menja listu? |
|------------------------------|-------------------------------|--------------|
| <code>append(x)</code>       | dodaje element na kraj        | da           |
| <code>extend(lista)</code>   | dodaje elemente druge liste   | da           |
| <code>lista1 + lista2</code> | pravi novu, spojenu listu     | ne           |
| <code>reverse()</code>       | obrće redosled                | da           |
| <code>sort()</code>          | sortira: abecedno ili rastuće | da           |
| <code>len(lista)</code>      | broj elemenata                | ne           |

```
apes.append("Pan paniscus")
primates = apes + monkeys
ranks = ["kingdom", "phylum", "class",
         "order", "family"]
ranks.reverse()
print(ranks)
ranks.sort()
print(ranks)
```

IZLAZ

```
['family', 'order', 'class', 'phylum', 'kingdom']
['class', 'family', 'kingdom', 'order', 'phylum']
```

# Isti posao bez petlje i sa petljom

## Bez petlje

```
print(apes[0] + " is an ape")  
print(apes[1] + " is an ape")  
print(apes[2] + " is an ape")
```

Ponavlja se, i moramo unapred da znamo koliko elemenata lista ima.

## Sa petljom

```
for ape in apes:  
    print(ape + " is an ape")
```

„Za svaki element liste apes, ispiši element i reči ‘is an ape’.”  
Radi za bilo koliko elemenata.

### IZLAZ OBA

```
Homo sapiens is an ape  
Pan troglodytes is an ape  
Gorilla gorilla is an ape
```

Sa petljom se izmena pravi na jednom mestu, a dodavanje vrsta u listu ne zahteva nikakvu izmenu petlje.

# Petlja for: ponavljanje sa varijacijom

```
apes = ["Homo sapiens",  
        "Pan troglodytes",  
        "Gorilla gorilla"]  
for ape in apes:  
    print(ape + " is an ape")
```

```
Homo sapiens is an ape  
Pan troglodytes is an ape  
Gorilla gorilla is an ape
```

IZLAZ

## Anatomija petlje

- **for x in y:** y je lista, a x redom dobija svaki element
- Prva linija se završava dvotačkom, a telo petlje je uvučeni blok
- Telo se izvršava jednom za svaki element
- Promenljiva petlje postoji samo unutar petlje (opseg, scope)

Sve linije bloka moraju biti uvučene isto, inače **IndentationError**. Ne mešajte tabove i razmake.

# Telo petlje može imati više naredbi

```
for ape in apes:  
    name_length = len(ape)  
    first_letter = ape[0]  
    print(ape + " is an ape. Its name starts with " + first_letter)  
    print("Its name has " + str(name_length) + " letters")
```

## IZLAZ

```
Homo sapiens is an ape. Its name starts with H  
Its name has 12 letters  
Pan troglodytes is an ape. Its name starts with P  
Its name has 15 letters  
Gorilla gorilla is an ape. Its name starts with G  
Its name has 15 letters
```

Četiri naredbe u telu, dve od njih print: dve linije izlaza po elementu.

U telu je dozvoljeno sve, osim menjanja liste kroz koju petlja prolazi.

# Blokovi i greške u uvlačenju

```
for ape in apes:
    name_length = len(ape)
    first_letter = ape[0]
    print(ape + " is an ape")
```

`IndentationError: unindent does not match any outer indentation level`

Treća linija je uvučena dva razmaka umesto četiri. Greška se javlja pre nego što program uopšte počne.

## Blok

Linija koja se završava dvotačkom, pa uvučene linije. Isti obrazac imaju petlje, funkcije (pogl. 5) i uslovi (pogl. 6). U drugim jezicima blok obeležavaju vitičaste zagrade.

Sve linije bloka moraju biti uvučene isto. Tab emulacija u editoru sprečava mešanje tabova i razmaka.

# Stringovi, fajlovi i split kao liste

## String: karakteri

```
name = "martin"  
for character in name:  
    print(character)
```

Svaki karakter je jedan element.  
Ponavljanje nad elementima zove se iteracija.

## Fajl: linije

```
file = open("input.txt")  
for line in file:  
    print(line)
```

Svaka linija je jedan element, zajedno sa znakom za novi red na kraju.

## split: nova lista

```
names = "melanogaster,simulans"  
species = names.split(",")  
print(species)
```

Deli string na svakom graničniku (delimiter): ['melanogaster', 'simulans']

Ne mešajte `read()` i petlju nad istim fajl objektom: posle `read()` Python je već na kraju fajla, pa petlja ne dobija nijednu liniju.

# Petlje sa opsezima: range

| Poziv                        | Brojevi     |
|------------------------------|-------------|
| <code>range(6)</code>        | 0 1 2 3 4 5 |
| <code>range(3, 8)</code>     | 3 4 5 6 7   |
| <code>range(2, 14, 4)</code> | 2 6 10      |

- Jedan broj: od 0 do tog broja, bez njega
- Dva broja: početak uključen, kraj isključen, kao kod isecanja
- Treći broj je korak

```
protein = "vlspadktnv"  
for stop in range(3, len(protein) + 1):  
    print(protein[0:stop])
```

```
vls  
vlsp  
vlspa  
vlspad  
vlspadk  
vlspadkt  
vlspadktn  
vlspadktnv
```

IZLAZ

# Korak 1: čitanje i isecanje

```
file = open("input.txt")
for dna in file:
    print(dna)
```

```
ATTCGATTATAAGCTCGATCGATCGATCGATCGATC...
ATTCGATTATAAGCACTGATCGATCGATCGATCGAT...
```

Prazni redovi: linija iz fajla već ima  
\n, a print dodaje još jedan. Za sada  
to ignorišemo.

```
file = open("input.txt")
for dna in file:
    last_character_position = len(dna)
    trimmed_dna = dna[14:last_character_position]
    print(trimmed_dna)
```

```
TCGATCGATCGATCGATCGATC...
ACTGATCGATCGATCGATCGAT...
ATCGATCACGATCTATCGTACG...
```

## Korak 2: upis u fajl i ispis dužina

```
file = open("input.txt")
output = open("trimmed.txt", "w")
for dna in file:
    last_character_position = len(dna)
    trimmed_dna = dna[14:last_character_position]
    output.write(trimmed_dna)
    print("processed sequence with length " + str(len(trimmed_dna)))
```

Izlazni fajl otvaramo pre petlje, a pišemo u njega unutar petlje.

Novi red ovde ne smeta: ostaje na kraju svake sekvence, pa su linije u fajlu tačno razdvojene.

Pažnja: ispisana dužina broji i \n. Za tačan broj baza prvo `rstrip("\n")`.

# Korak 1: čitanje pozicija egzona

EXONS.TXT

5,58  
72,133  
190,276  
340,398

```
exon_locations = open("exons.txt")  
for line in exon_locations:  
    positions = line.split(',')  
    print(positions)
```

```
['5', '58\n']  
['72', '133\n']  
['190', '276\n']  
['340', '398\n']
```

```
# u petlji, posle split:  
start = positions[0]  
stop = positions[1]  
print("start is " + start + ", stop is " + stop)
```

start is 5, stop is 58

start is 72, stop is 133

## Korak 2: string nije broj

```
genomic_dna = open("genomic_dna.txt").read()
exon_locations = open("exons.txt")
for line in exon_locations:
    positions = line.split(',')
    start = positions[0]
    stop = positions[1]
    exon = genomic_dna[start:stop]
    print("exon is: " + exon)
```

TypeError: slice indices must be integers or None or have an `__index__` method

`start` i `stop` su stringovi, jer su elementi liste koju je vratio `split`.

```
start = int(positions[0])
stop = int(positions[1])
```

`int` pretvara string u ceo broj; `int('58\n')` daje 58, jer razmaci i novi red oko broja ne smetaju.

## Korak 3: spajanje egzona i upis u fajl

```
genomic_dna = open("genomic_dna.txt").read()
exon_locations = open("exons.txt")
coding_sequence = ""
for line in exon_locations:
    positions = line.split(',')
    start = int(positions[0])
    stop = int(positions[1])
    exon = genomic_dna[start:stop]
    coding_sequence = coding_sequence + exon
output = open("coding_sequence.txt", "w")
output.write(coding_sequence)
output.close()
```

U `exons.txt` je jedan egzon po liniji, u obliku `početak,kraj`.

**1** `split(',')` deli liniju na dva stringa

**2** `int()` pretvara string u broj, jer pozicije za isecanje moraju biti brojevi

**3** Promenljiva je sa obe strane znaka `=:` prvo se računa desna strana, pa se rezultat upisuje nazad

# Rezime i vežbe

## Šta smo naučili

- Liste: pravljenje, indeksi (i negativni), isecanje
- append, extend, reverse, sort i spajanje sa +
- Petlja for: dvotačka, uvučeni blok, promenljiva petlje
- Uvlačenje mora biti dosledno: IndentationError
- Stringovi, fajlovi i range ponašaju se kao liste
- split pravi listu od stringa

## Vežbe

1. U input.txt svaka sekvenca počinje istim adapterom od 14 baza. Uklonite ga, upišite očišćene sekvence u novi fajl i ispišite dužinu svake
2. Iz genomic\_dna.txt izdvojite egzone čije su pozicije u exons.txt, spojite ih i upišite u novi fajl